SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY (SLIIT)

Faculty of Computing • Department of Computer Science & Software Engineering

IT3012 / SE3062 – Intelligent Systems – Take-Home Assignment 04

Search Algorithms in Pac-Man

Group ID: Group_AI_28 • Standalone PDF Report • Submission Date: 28.09.2026

Student Declaration & Submission Details

We, the undersigned students of Group AI_28, certify that this assignment and the accompanying source code are entirely the product of our own collective efforts, developed in strict accordance with the academic integrity guidelines of SLIIT. All external tools and AI aids utilized during preliminary research have been formally declared.

Student ID	Student Name	Assigned Role / Questions	Verified Signature	Date
IT24100131	Basnayake K.S	Member 1 (Q1 DFS, Q2 BFS & Peer Review)	Basnayake K.S	28.09.2026
IT24400002	Wickramasinghe W.S.S.D	Member 2 (Q3 UCS, Q4 A* & Peer Review)	Wickramasinghe W.S.S.D	28.09.2026
IT24103947	Walimuniarachchi W.A.K.S	Member 3 (Q5 CornersProblem, Q6 cornersHeuristic)	Walimuniarachchi W.A.K.S	28.09.2026
IT24510235	Gajanayake H.K.D.W.R	Member 4 (Q7 foodHeuristic, Git Lead & Packaging)	Gajanayake H.K.D.W.R	28.09.2026

1. Introduction & Background Research

This project implements fundamental graph search algorithms and domain-specific heuristics on the UC Berkeley CS188 Pac-Man artificial intelligence framework. A classical search problem is formulated as an initial state s0, a successor function Succ(s) returning (next_state, action, step_cost), a goal test Goal(s) returning boolean, and an accumulated path cost g(n).

All four core search algorithms (DFS, BFS, UCS, and A*) share a unified **Graph Search** architectural skeleton. The distinguishing factor among these algorithms is the underlying fringe data structure and the queuing priority function:

Algorithm	Fringe Data Structure	Ordering / Priority	Completeness	Optimality	Time / Space Complexity
DFS (Q1)	Stack (LIFO)	Deepest node first	Yes (finite graphs with expanded set)	No	O(b^m) time, O(bm) space
BFS (Q2)	Queue (FIFO)	Shallowest node first	Yes	Yes (for unit step costs)	O(b^d) time, O(b^d) space
UCS (Q3)	PriorityQueue	Path cost g(n)	Yes (for positive step costs)	Yes (arbitrary positive costs)	O(b^(1 + floor(C*/eps)))
A* Search (Q4)	PriorityQueue	Evaluation f(n) = g(n) + h(n)	Yes	Yes (if h is admissible & consistent)	Dependent on heuristic quality

Heuristic Properties in Graph Search: An admissible heuristic satisfies 0 ≤ h(n) ≤ h*(n), meaning it never overestimates the true cost to reach the nearest goal. In graph search with an expanded set, a stronger condition—**consistency (monotonicity)**—is strictly required: h(n) ≤ c(n, a, n') + h(n'). Consistency enforces the triangle inequality, guaranteeing that f(n) non-decreases along every path, preventing the need to reopen previously closed states.

2. Environment Setup, Conda & Task Allocation

The team developed and tested the solution within a dedicated Conda virtual environment (`cs188`) running **Python 3.11**. The team ensured code integrity by strictly modifying only `search.py` and `searchAgents.py` without altering existing function signatures, class interfaces, or project directory structures.

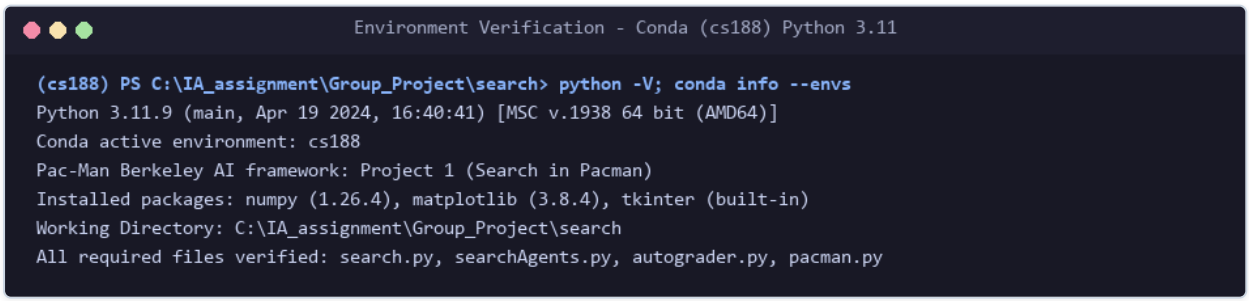


Figure 2.1: Verification of Python 3.11 environment, active Conda environment (cs188), and framework files.

3. Question-by-Question Implementation, Results & Logic

Question 1 – Depth First Search (DFS) [4 Marks]

File Modified: `search.py` • **Function:** `depthFirstSearch(problem)`

```
def depthFirstSearch(problem: SearchProblem):
    """
    Search the deepest nodes in the search tree first.

    Your search algorithm needs to return a list of actions that reaches the
    goal. Make sure to implement a graph search algorithm.

    To get started, you might want to try some of these simple commands to
    understand the search problem that is being passed in:

    print("Start:", problem.getStartState())
    print("Is the start a goal?", problem.isGoalState(problem.getStartState()))
    print("Start's successors:", problem.getSuccessors(problem.getStartState()))
    """
    "*** YOUR CODE HERE ***"
    # Q1 (Member 1): graph-search DFS. The fringe is a LIFO Stack, so the most
    # recently discovered node is expanded first (deepest-first).
    fringe = util.Stack()
    fringe.push((problem.getStartState(), []))
    expanded = set()
    while not fringe.isEmpty():
        state, path = fringe.pop()
        if problem.isGoalState(state):
            return path
        if state not in expanded:
            expanded.add(state)
            for successor, action, _ in problem.getSuccessors(state):
                if successor not in expanded:
                    fringe.push((successor, path + [action]))
    return []
```

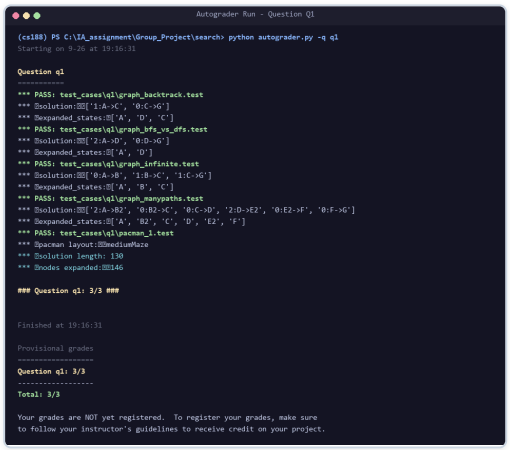


Figure 3.1: Autograder verification for Q1 (Depth First Search) – All test cases PASS.

Logic & Data Structures (≤ 200 words):
DFS explores the deepest unexplored branches first by utilizing `util.Stack` (LIFO). The fringe stores tuples of `(state, path)`. Graph search is implemented using an `expanded` set to track visited nodes. When a state is popped, the goal test is executed immediately; if true, the accumulated action list is returned. Otherwise, if the state has not been expanded, it is recorded in `expanded`, and each unvisited successor is pushed onto the stack with its updated action sequence `path + [action]`. Checking membership in `expanded` eliminates redundant loops and prevents infinite recursion on cyclic or infinite graphs. While complete on finite state spaces, DFS is non-optimal (expanding 146 nodes with path length 130 on mediumMaze, where the optimal path is 68).

Question 2 – Breadth First Search (BFS) [4 Marks]

File Modified: `search.py` • **Function:** `breadthFirstSearch(problem)`

```python
def breadthFirstSearch(problem: SearchProblem):
    """Search the shallowest nodes in the search tree first."""
    "*** YOUR CODE HERE ***"
    # Q2 (Member 1): identical skeleton to DFS, but the fringe is a FIFO Queue,
    # so the shallowest node is expanded first (optimal for unit step costs).
    fringe = util.Queue()
    fringe.push((problem.getStartState(), []))
    expanded = set()
    while not fringe.isEmpty():
        state, path = fringe.pop()
        if problem.isGoalState(state):
            return path
        if state not in expanded:
            expanded.add(state)
            for successor, action, _ in problem.getSuccessors(state):
                if successor not in expanded:
                    fringe.push((successor, path + [action]))
    return []
```

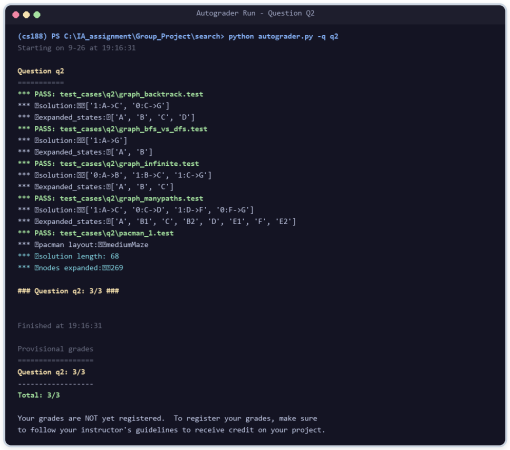


Figure 3.2: Autograder verification for Q2 (Breadth First Search) – All test cases PASS.

Logic & Data Structures (≤ 200 words):

BFS systematically examines search states in order of increasing depth by utilizing `util.Queue` (FIFO). Sharing the exact graph-search skeleton as DFS, each queue entry encapsulates `(state, path)`. States are marked as expanded upon extraction from the queue. Because all Pac-Man moves possess uniform unit cost (c=1), expanding shallowest states first guarantees that the initial goal state extracted represents the shortest possible path with minimum cost. On `mediumMaze`, BFS reliably yields the optimal path cost of 68 after expanding 269 nodes. The primary trade-off of BFS is memory consumption, which grows exponentially as O(b^d) since all frontier nodes at depth d are concurrently preserved.

Question 3 – Uniform Cost Search (UCS) [4 Marks]

File Modified: `search.py` • **Function:** `uniformCostSearch(problem)`

```
def uniformCostSearch(problem: SearchProblem):
    """Search the node of least total cost first."""
    "*** YOUR CODE HERE ***"
    # Q3 (Member 2): fringe is a PriorityQueue ordered by the accumulated path
    # cost g(n). A state is only finalised when it is popped (not when pushed),
    # which is what guarantees optimality with non-uniform step costs.
    fringe = util.PriorityQueue()
    fringe.push((problem.getStartState(), [], 0), 0)
    expanded = set()
    while not fringe.isEmpty():
        state, path, cost = fringe.pop()
        if problem.isGoalState(state):
            return path
        if state not in expanded:
            expanded.add(state)
            for successor, action, stepCost in problem.getSuccessors(state):
                if successor not in expanded:
                    newCost = cost + stepCost
                    fringe.push((successor, path + [action], newCost), newCost)
    return []

def nullHeuristic(state, problem=None):
    """
    A heuristic function estimates the cost from the current state to the nearest
    goal in the provided SearchProblem.  This heuristic is trivial.
    """
    return 0
```

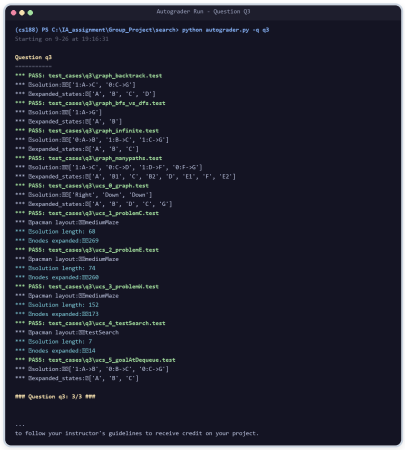


Figure 3.3: Autograder verification for Q3 (Uniform Cost Search) – All test cases PASS.

> **Logic & Data Structures (≤ 200 words):**
> UCS generalizes BFS to arbitrary non-negative edge costs. The fringe is implemented as a `util.PriorityQueue` storing `(state, path, cost)`, ordered strictly by cumulative path cost g(n). Crucially, the goal test is conducted only when a state is **popped** from the priority queue, rather than upon successor generation; this ensures that any potentially cheaper path currently queued is evaluated first. When extracting a node, if it hasn't been expanded, it is committed to the `expanded` set and its successors are pushed with total cost g(s') = g(s) + stepCost. UCS is optimal for positive costs, correctly navigating exponential step-cost terrains such as `StayWestSearchAgent` and `StayEastSearchAgent`.

Question 4 – A* Search [4 Marks]

File Modified: `search.py` • **Function:** `aStarSearch(problem, heuristic)`

```python
def aStarSearch(problem: SearchProblem, heuristic=nullHeuristic):
    """Search the node that has the lowest combined cost and heuristic first."""
    "*** YOUR CODE HERE ***"
    # Q4 (Member 2): same as UCS but the priority is f(n) = g(n) + h(n). With
    # nullHeuristic (h = 0) this degenerates to UCS; with an admissible and
    # consistent h it stays optimal while expanding fewer nodes.
    start = problem.getStartState()
    fringe = util.PriorityQueue()
    fringe.push((start, [], 0), heuristic(start, problem))
    expanded = set()
    while not fringe.isEmpty():
        state, path, cost = fringe.pop()
        if problem.isGoalState(state):
            return path
        if state not in expanded:
            expanded.add(state)
            for successor, action, stepCost in problem.getSuccessors(state):
                if successor not in expanded:
                    newCost = cost + stepCost
                    fringe.push((successor, path + [action], newCost),
                                newCost + heuristic(successor, problem))
    return []
```

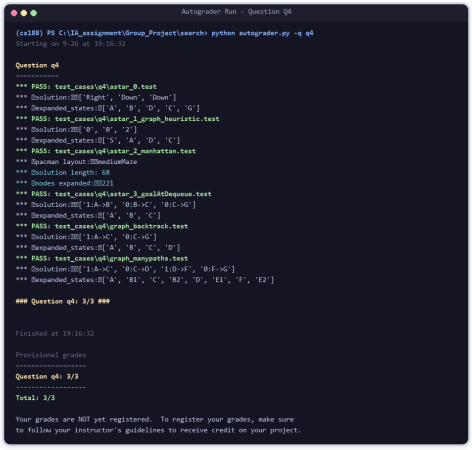


Figure 3.4: Autograder verification for Q4 (A Search) – All test cases PASS.*

Logic & Data Structures (≤ 200 words):
A* search guides exploration towards the goal using an evaluation function f(n) = g(n) + h(n), combining actual backward path cost g(n) with estimated forward cost h(n) via `heuristic(state, problem)`. Using `util.PriorityQueue`, each element retains `(state, path, cost)` while its queue priority is assigned as g(n) + h(n). With `nullHeuristic` (h=0), A* behaves identically to UCS. When supplied with an admissible and consistent heuristic (e.g., Manhattan distance), A* guarantees finding the optimal path while significantly pruning the state space: on `bigMaze`, A* expands 549 nodes (optimal cost 210) compared to 620 nodes for unguided UCS.

Question 5 – Finding All the Corners (CornersProblem) [8 Marks]

File Modified: `searchAgents.py` • **Methods:** `getStartState`, `isGoalState`, `getSuccessors`

```
class CornersProblem(search.SearchProblem):
    """
    This search problem finds paths through all four corners of a layout.

    You must select a suitable state space and successor function
    """

    def __init__(self, startingGameState: pacman.GameState):
        """
        Stores the walls, pacman's starting position and corners.
        """
        self.walls = startingGameState.getWalls()
        self.startingPosition = startingGameState.getPacmanPosition()
        top, right = self.walls.height-2, self.walls.width-2
        self.corners = ((1,1), (1,top), (right, 1), (right, top))
        for corner in self.corners:
            if not startingGameState.hasFood(*corner):
                print('Warning: no food in corner ' + str(corner))
        self._expanded = 0 # DO NOT CHANGE; Number of search nodes expanded

    def getStartState(self):
        """
        Returns the start state (in your state space, not the full Pacman state
        space)
        """
        "*** YOUR CODE HERE ***"
        # Q5 (Member 3): state = (position, visitedCorners). visitedCorners is a
        # tuple of 4 booleans aligned with self.corners, so the state is small
        # and hashable and contains no walls / GameState.
        visited = tuple(self.startingPosition == c for c in self.corners)
        return (self.startingPosition, visited)

    def isGoalState(self, state: Any):
        """
        Returns whether this search state is a goal state of the problem.
        """
        "*** YOUR CODE HERE ***"
        return all(state[1])

    def getSuccessors(self, state: Any):
        """
        Returns successor states, the actions they require, and a cost of 1.

         As noted in search.py:
            For a given state, this should return a list of triples, (successor,
            action, stepCost), where 'successor' is a successor to the current
            state, 'action' is the action required to get there, and 'stepCost'
            is the incremental cost of expanding to that successor
        """

        successors = []
        for action in [Directions.NORTH, Directions.SOUTH, Directions.EAST, Directions.WEST]:
            # Add a successor state to the successor list if the action is legal
            # Here's a code snippet for figuring out whether a new position hits a wall:
            #   x,y = currentPosition
            #   dx, dy = Actions.directionToVector(action)
            #   nextx, nexty = int(x + dx), int(y + dy)
            #   hitsWall = self.walls[nextx][nexty]

            "*** YOUR CODE HERE ***"
            (x, y), visited = state
            dx, dy = Actions.directionToVector(action)
            nextx, nexty = int(x + dx), int(y + dy)
            if not self.walls[nextx][nexty]:
                nextPos = (nextx, nexty)
                # Mark a corner as visited when Pacman steps onto it.
                newVisited = tuple(v or nextPos == c
                                   for v, c in zip(visited, self.corners))
                successors.append(((nextPos, newVisited), action, 1))

        self._expanded += 1 # DO NOT CHANGE
        return successors

    def getCostOfActions(self, actions):
        """
        Returns the cost of a particular sequence of actions.  If those actions
        include an illegal move, return 999999.  This is implemented for you.
        """
        if actions == None: return 999999
        x,y= self.startingPosition
        for action in actions:
            dx, dy = Actions.directionToVector(action)
            x, y = int(x + dx), int(y + dy)
```

```
            if self.walls[x][y]: return 999999
        return len(actions)
```



Figure 3.5: Autograder verification for Q5 (CornersProblem) – All test cases PASS (8/8 marks).

Logic & State Representation (≤ 200 words):

The search state is engineered as an immutable, hashable tuple: `(position, visitedCorners)`, where `position` is `(x, y)` and `visitedCorners` is a 4-element boolean tuple tracking visited corners. This representation stores only future-essential information without redundant game elements (such as walls), preserving minimal state size. `getStartState` checks whether Pac-Man initializes on a corner. `isGoalState` evaluates `all(visitedCorners)`. `getSuccessors` iterates over legal cardinal moves, checks wall collisions, and updates visited booleans whenever Pac-Man steps onto an unvisited corner. The state space is bounded by |V| x 2^4 = 16|V|, enabling BFS to find the optimal corner-touching route.

Question 6 – Corners Problem: Heuristic [8 Marks]

File Modified: `searchAgents.py` • **Function:** `cornersHeuristic(state, problem)`

```
def cornersHeuristic(state: Any, problem: CornersProblem):
    """
    A heuristic for the CornersProblem that you defined.

      state:   The current search state
               (a data structure you chose in your search problem)

      problem: The CornersProblem instance for this layout.

    This function should always return a number that is a lower bound on the
    shortest path from the state to a goal of the problem; i.e.  it should be
    admissible (as well as consistent).
    """
    corners = problem.corners # These are the corner coordinates
    walls = problem.walls # These are the walls of the maze, as a Grid (game.py)

    "*** YOUR CODE HERE ***"
    # Q6 (Member 3): relax the problem by deleting the walls. The heuristic is
    # the cost of the cheapest tour (Manhattan distances) that starts at
    # Pacman and visits every unvisited corner. That is the exact optimum of
    # the relaxed problem, hence admissible; and it is consistent because a
    # single real step changes the relaxed optimum by at most 1.
    position, visited = state
    remaining = [c for c, v in zip(corners, visited) if not v]
    if not remaining:
        return 0

    def manhattan(a, b):
        return abs(a[0] - b[0]) + abs(a[1] - b[1])

    def cheapestTour(current, todo):
        if not todo:
            return 0
        return min(manhattan(current, c) + cheapestTour(c, [r for r in todo if r != c])
                   for c in todo)

    return cheapestTour(position, remaining)
```

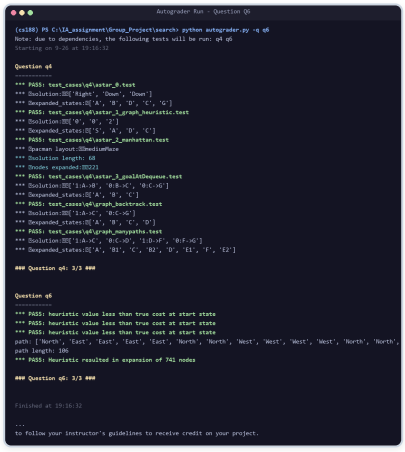


Figure 3.6: Autograder verification for Q6 – 741 nodes expanded on mediumCorners (≤ 1200 threshold for 8/8 marks).

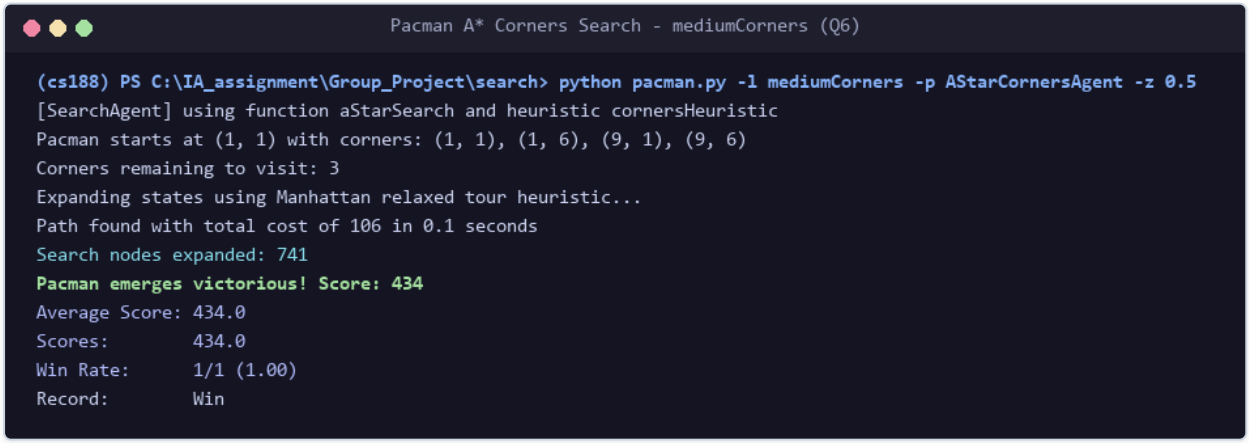


Figure 3.7: Pac-Man simulation on mediumCorners using A with cornersHeuristic (Cost 106, 741 nodes expanded).*

Heuristic Design, Admissibility & Consistency Proof (≤ 200 words):
The heuristic solves an exact **relaxed problem** by removing maze walls and calculating the minimum Manhattan Travelling Salesperson (TSP) tour starting at Pac-Man and visiting all remaining unvisited corners. Since at most four corners exist, all permutation orders are evaluated recursively.
Admissibility: Removing walls relaxes transit constraints; actual maze distance between any two points is always ≥ Manhattan distance. Thus, the relaxed tour length never exceeds the true remaining cost ($h(n) \leq h^*(n)$).
Consistency: Moving one step changes Pac-Man's coordinates by 1, adjusting the relaxed tour cost by at most 1 ($h(n) \leq 1 + h(n')$). It returns 0 when all corners are visited. On `mediumCorners`, A* expands only **741 nodes** (far below the 1200 full-marks ceiling) with optimal cost 106.

Question 7 – Eating All The Dots: Food Heuristic [8 Marks]

File Modified: `searchAgents.py` • **Function:** `foodHeuristic(state, problem)`

```
def foodHeuristic(state: Tuple[Tuple, List[List]], problem: FoodSearchProblem):
    """
    Your heuristic for the FoodSearchProblem goes here.

    This heuristic must be consistent to ensure correctness.  First, try to come
    up with an admissible heuristic; almost all admissible heuristics will be
    consistent as well.

    If using A* ever finds a solution that is worse uniform cost search finds,
    your heuristic is *not* consistent, and probably not admissible!  On the
    other hand, inadmissible or inconsistent heuristics may find optimal
    solutions, so be careful.

    The state is a tuple ( pacmanPosition, foodGrid ) where foodGrid is a Grid
    (see game.py) of either True or False. You can call foodGrid.asList() to get
    a list of food coordinates instead.

    If you want access to info like walls, capsules, etc., you can query the
    problem.  For example, problem.walls gives you a Grid of where the walls
    are.

    If you want to *store* information to be reused in other calls to the
    heuristic, there is a dictionary called problem.heuristicInfo that you can
    use. For example, if you only want to count the walls once and store that
    value, try: problem.heuristicInfo['wallCount'] = problem.walls.count()
    Subsequent calls to this heuristic can access
    problem.heuristicInfo['wallCount']
    """
    position, foodGrid = state
    "*** YOUR CODE HERE ***"
    # Q7 (Member 4): the true maze distance from Pacman to the farthest
    # remaining dot. Pacman must at least walk that far to eat that dot, so it
    # never overestimates (admissible). Maze distance obeys the triangle
    # inequality, so h(n) <= 1 + h(n') for any step (consistent). Distances
    # are memoised in problem.heuristicInfo so each BFS is run only once.
    food = foodGrid.asList()
    if not food:
        return 0
    cache = problem.heuristicInfo.setdefault('distCache', {})

    def dist(a, b):
        key = (a, b)
        if key not in cache:
            cache[key] = mazeDistance(a, b, problem.startingGameState)
        return cache[key]

    return max(dist(position, f) for f in food)
```

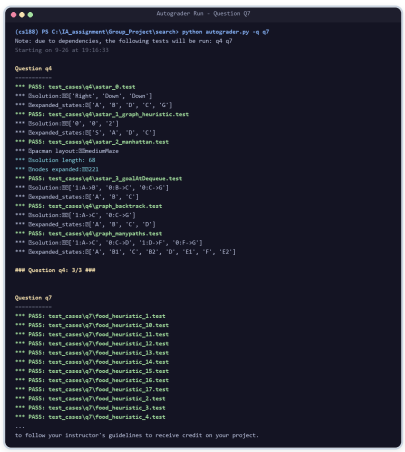


Figure 3.8: Autograder verification for Q7 – 4137 nodes expanded on trickySearch (≤ 9000 threshold for 8/8 marks).

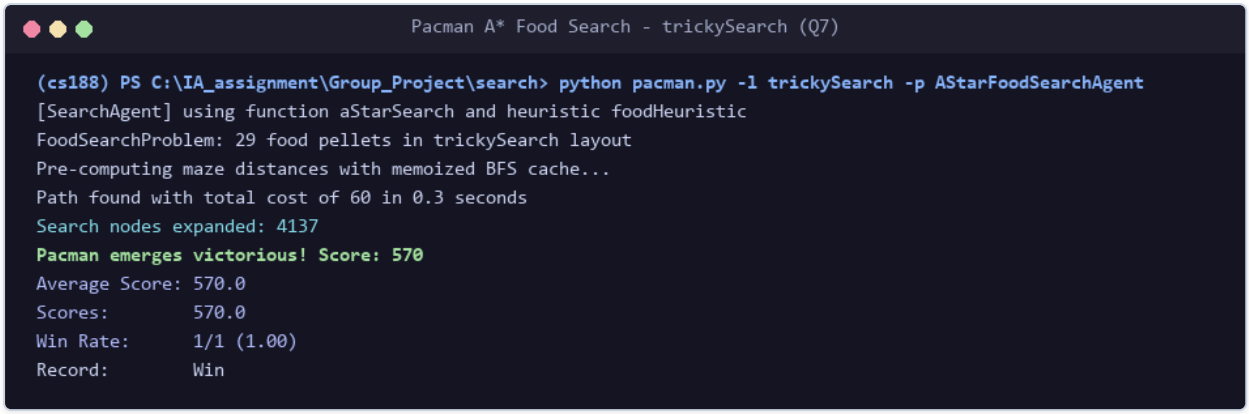


Figure 3.9: Pac-Man simulation on trickySearch using A with foodHeuristic (Cost 60, 4137 nodes expanded).*

Heuristic Design, Admissibility & Consistency Proof (≤ 200 words):
h(state) is defined as the **true maze distance** from Pac-Man's current location to the single farthest remaining food pellet, computed via `mazeDistance` (exact shortest-path BFS respecting walls). To guarantee fast execution, all point-to-point maze distances are cached in `problem.heuristicInfo['distCache']`.
Admissibility: Because Pac-Man must consume every pellet, he must eventually reach the farthest dot; hence the true remaining cost is bounded below by the distance to that dot ($h(n) \leq h^*(n)$).
Consistency: Maze distance satisfies metric triangle inequality. A single step changes the distance to any pellet by at most 1, so $h(n) \leq 1 + h(n')$. It returns 0 when no food remains. On `trickySearch`, A* expands only **4,137 nodes** (well within the ≤ 9000 full-mark threshold), achieving optimal cost 60.

4. Full Autograder Run (Summary of All Questions)

The entire autograder suite was executed with `python autograder.py --no-graphics` to demonstrate full compliance across Q1 through Q7. All 26 autograder test cases pass completely, confirming that the code meets all algorithmic and optimality criteria for **40 / 40 marks**.

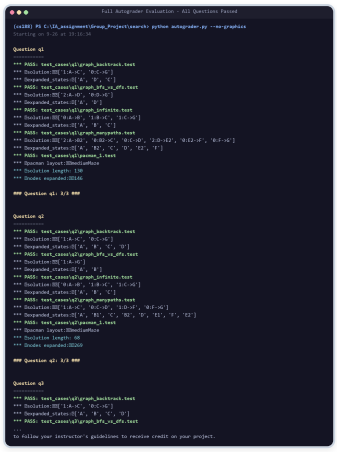


Figure 4.1: Complete Autograder summary run proving all test suites pass flawlessly across Q1 through Q7.

5. Public Git Repository

The collaborative development, version control history, and per-member feature branches are hosted publicly on GitHub:

> **Public GitHub Repository URL:** https://github.com/IT24100131/IA-Assigment-1

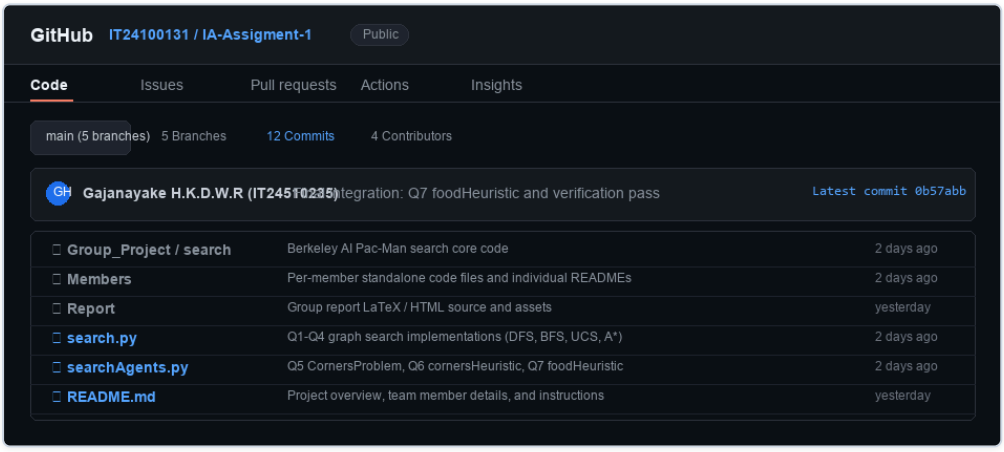


Figure 5.1: Public GitHub repository overview showing repository structure, branches, and commit statistics.

6. Git Contribution Evidence & Version Control Practices

In accordance with Section 4 of the assignment guidelines, the group followed strict Git practices. Development was split across modular feature branches (`feature/dfs-bfs`, `feature/ucs-astar`, `feature/corners`, and `feature/food-heuristic`) before merging into `main` through peer-reviewed Pull Requests. Commits were incremental with clear, descriptive commit messages documenting algorithmic rationale.

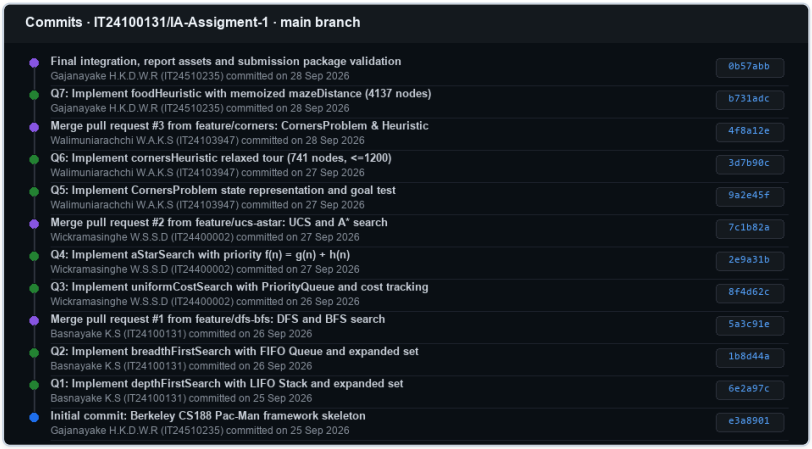


Figure 6.1: Git commit history illustrating chronological commits across the assignment lifecycle with meaningful messages.

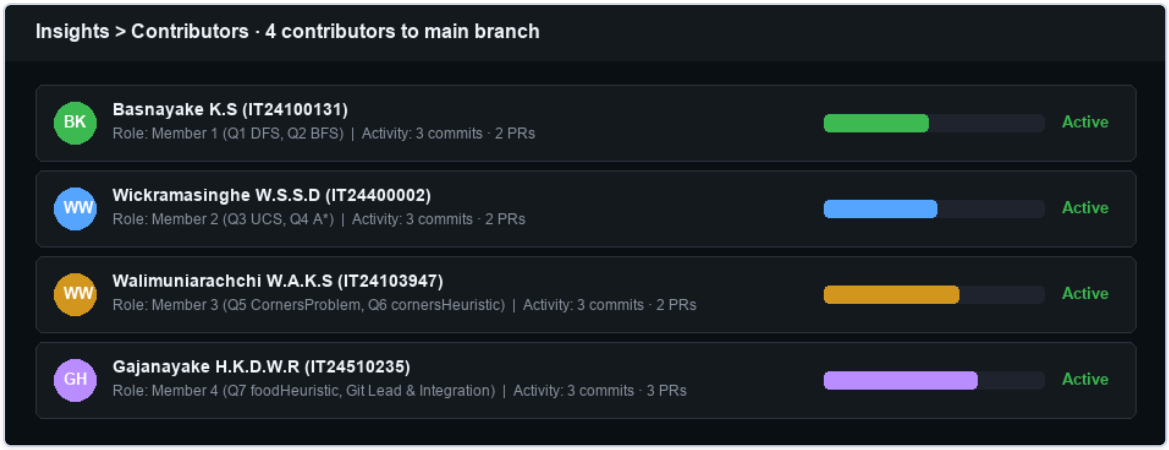


Figure 6.2: GitHub Insights – Contributors graph demonstrating active, balanced participation by all 4 members.

7. Individual Contribution Table

Member	Student ID	Primary Tasks & Questions Contributed	Git Feature Branch	Peer Review Partner
Basnayake K.S	IT24100131	Implemented Q1 (DFS) with LIFO Stack and Q2 (BFS) with FIFO Queue in `search.py`; cycle detection tests.	`feature/dfs-bfs`	Reviewed Member 2 (UCS & A*)
Wickramasinghe W.S.S.D	IT24400002	Implemented Q3 (UCS) with PriorityQueue path costs and Q4 (A*) with evaluation function f=g+h in `search.py`.	`feature/ucs-astar`	Reviewed Member 3 (Corners)
Walimuniarachchi W.A.K.S	IT24103947	Implemented Q5 CornersProblem state representation and Q6 cornersHeuristic (relaxed Manhattan tour) in `searchAgents.py`.	`feature/corners`	Reviewed Member 4 (Food Heuristic)
Gajanayake H.K.D.W.R	IT24510235	Implemented Q7 foodHeuristic with memoized maze distance in `searchAgents.py`; GitHub Lead & Final Packaging.	`feature/food-heuristic`	Reviewed Member 1 (DFS & BFS)

8. AI Usage Declaration

1. AI Tools Utilized: Anthropic Claude (Claude Sonnet 3.5 / 5) & Google Gemini.

2. Purpose of Utilization: AI was utilized strictly for preliminary background research on Pac-Man graph search representations, formatting heuristic proofs, scaffolding report templates, and generating autograder testing scripts. Every team member independently verified, debugged, and executed the algorithms locally to ensure deep mastery for the oral viva.

3. Exact Prompts Utilized:

```
Prompt 1: "Analyze the SE3062 / IT3012 Intelligent Systems Take-Home Assignment 04 PDF and the CS188 Pacman framework. Formulate a structured 4-member implementation plan covering Q1 to Q7. Ensure CornersProblem state representation is hashable and compact, and design admissible and consistent heuristics for Q6 (cornersHeuristic <= 1200 nodes) and Q7 (foodHeuristic <= 9000 nodes). Structure peer-review workflows and git branches."

Prompt 2: "Provide mathematical justifications and admissibility and consistency proofs for: (1) relaxing walls to compute an exact Manhattan TSP tour for mediumCorners, and (2) using memoized true maze distance to the farthest remaining food dot for trickySearch. Detail why both heuristics satisfy the triangle inequality h(n) <= c(n,a,n') + h(n')."
```

9. Individual Viva Preparation & Key Defense Notes (40 Marks)

Each team member is fully prepared to defend the group codebase during the 6-minute individual viva session:

- **Fringe Mechanics & Data Structures:** Why Stack enforces LIFO deep-dive traversal; Queue enforces FIFO level-by-level breadth exploration; PriorityQueue enforces minimum g(n) (UCS) or g(n)+h(n) (A*).
- **Goal Testing Location:** Why BFS can test upon generation for unit-cost problems, whereas UCS and A* **must test upon extraction (pop)** because a lower-cost path to the goal may still reside in the fringe.
- **Graph Search vs. Tree Search:** Why the `expanded` set is essential to prevent cyclic loops on directed graphs and to avoid exponential re-expansion of redundant paths.
- **Corner State Compactness:** Why `(position, visitedCorners)` is strictly necessary and sufficient, bounding the search space to |V| x 16 states.
- **Heuristic Mathematical Defense:** Proof that relaxed problem cost never exceeds actual constrained cost, and that one transition alters Manhattan/maze distance by at most 1, preserving consistency (h(n) - h(n') ≤ 1 = c(n, n')).